

CMSC 471 - PA1: A* Search and Heuristics with DOGCAT

Name: Hassan Ansari

ID: FF82216

Section: 02

Sources / collaboration (cite external resources, mention anyone you discussed with):

-Geeksforgeeks python pages

Generative AI disclosure (if used outside Q4):

- System used: Claude code

- What I used it for: Debugging my [dc.py](#) and fix the heuristic function and checking if my outputs for the test were right. Also, it checked if my answers made sense.

- How I verified the result: I ran the sample commands from the assignment and compared the outputs to the PDF, and checked each heuristic against the true optimal remaining cost on many states with no violations.

QUESTION 1: Heuristic design and admissibility [15 pts]

For each of the three cost measures, describe the heuristic in your program.

For each heuristic: (1) precise formula or pseudocode, (2) what unavoidable remaining cost it lower-bounds, (3) why it cannot overestimate the true optimal remaining cost. Must be specific to the cost measure; just saying "it is a lower bound" is insufficient.

Steps

- Formula / pseudocode: $h = k$

k = number of positions that differ from the goal

- What unavoidable cost it lower-bounds: The minimum amount of moves needed.

- Why it never overestimates: Each moves changes a letter so it fixes one spot that is wrong and each move costs 1 so any solution of n moves costs at least n .

Scrabble

- Formula / pseudocode: h is the sum of $\text{value}(\text{goal}[i])$ over the mismatched positions.

- What unavoidable cost it lower-bounds: The cost of writing each letter that's missing

- Why it never overestimates: The last move that touches position i has to insert $\text{goal}[i]$, Different position need different moves to get to them so those cost add up without double counting and any new moves is extra cost

Frequency

- Formula / pseudocode: $h=0$ at the goal but otherwise $k + \text{rarity}(\text{goal})$

- What unavoidable cost it lower-bounds: at least k moves which the last one enters the goal word which costs $1 + \text{rarity}(\text{goal})$

- Why it never overestimates: Each k-1 moves $1 + \text{rarity}(w)$ which is at least 1 and so the actual cost is at least $k-1 + 1 + \text{rarity}(\text{goal})$

QUESTION 2: Does the heuristic actually help? [10 pts]

Run uniform-cost search and A* (display=True) on WHY -> ASK and BEAR -> DUCK with all three cost measures. Use a new DC instance for every search. Nodes expanded = number reported by the search function with display=True.

Reduction (%) = $100 * (\text{UCS expansions} - \text{A* expansions}) / \text{UCS expansions}$
(negative means A* expanded more nodes than UCS)

Word pair	Cost measure	UCS cost	UCS nodes	A* cost	A* nodes	Reduction
WHY -> ASK	Steps	5	773	5	16	97.9%
WHY -> ASK	Scrabble	9	765	9	13	98.3%
WHY -> ASK	Frequency	38.86	808	38.86	634	21.5%
BEAR -> DUCK	Steps	4	691	4	6	99.1%
BEAR -> DUCK	Scrabble	11	2981	11	6	99.8%
BEAR -> DUCK	Frequency	38.24	1084	38.24	302	72.1%

Answer the following:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.

Yea it always returned the same optimal cost in all of the runs which is what you'd expect.

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

- WHY -> ASK: Scrabble 98.3%
- BEAR -> DUCK: Scrabble 99.8%

3. For each word pair, under which cost measure was the heuristic least informative?

- WHY -> ASK: Frequency 21.5%
- BEAR -> DUCK: Frequency 72.1%

4. Explain the differences using the definitions of your heuristics (not runtime alone).

- Scrabble and steps told a lot and that their heuristics captured most of the true cost as steps counts the spots that are mismatched and scrabble gets the goal letter scrabble values. Frequency didn't tell as much because the cost ended up being heavily influenced by the words in the middle and the heuristic only adds $\text{rarity}(\text{goal})$ and ignores every other word.

QUESTION 3: Reversibility and direction-dependent costs [5 pts]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal. For each cost measure, determine whether the optimal cost from W1 to W2 must always equal the optimal cost from W2 to W1. For each: (1) yes or no, (2) justification, (3) if no, a concrete counterexample using words from the provided dictionary. Clearly distinguish reversibility of an action from symmetry of its cost.

Steps

- Yes / No: Yes
- Justification: Every move costs 1 and every action can be undone by an action . If you're taking an optimal path to get to point2 from point 1 reversing would get the same steps or in other words going from point2 to point1.
- Counterexample (if no):n/a

Scrabble

- Yes / No: No
- Justification: Basically every action costs the scrabble letter value of the letter being inserted and reverse action inserts the old letter. So, the same action could cost more or less going in reverse compared to normal.
- Counterexample (if no):
A simple counter example is that going from ask to why costs 14 as you have to change you go ask ass ahs aha wha why to total 14. However going the other way or why to ask costs 9 as the path why wha aha ahs ass ask has cheaper scrabble values this way.

Frequency

- Yes / No: No
- Justification: Each action ends up costing 1 + rarity(word) and a forward path pays for every word after the beginning including the end whereas the reverse path pays for every word before the goal including start which results in having different paths and values in the end because the sets of words are different unless the start and end have the same rarity value.
- Counterexample (if no):
One simple example going from ask to why because it would cost 37.76 with a path of :ask ass ais his has was way why whereas the other way around of why to ask would cost 38.86 with a path of why way was has his ais ass ask.

- Reversibility of an action vs. symmetry of its cost:
Symmetry of its cost is about costing the same both forwards and backwards and reversibility of an action is if the changes/ actions made are valid.

QUESTION 4: LLM heuristic audit [10 pts]

Prompt to give the generative AI system (use exactly this text):

In DOGCAT, the initial and goal states are lowercase words of the same length--three or four letters--and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

1. Model used, exact prompt, and the heuristic it proposed:

- Model: GPT-5.6 Luna
- Exact prompt used: In DOGCAT, the initial and goal states are lowercase words of the same length--three or four letters--and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

- Proposed heuristic (its response, summarized or quoted):

for each position i , let m_i be the minimum rarity of any dictionary word with the goal letter in position i , and let $h_R(n) = \sum \text{over mismatched } i \text{ of } (1 + m_i)$.

2. Precise formula or pseudocode for the proposed heuristic:

- $m_i = \min\{ \text{rarity}(w) : w \text{ in dictionary, } \text{len}(w) = \text{len}(\text{goal}), w[i] = \text{goal}[i] \}$.
- $h_R(n) = 0$ if n is the goal, otherwise $\sum_{i : n[i] \neq \text{goal}[i]} (1 + m_i)$. This equals $h_0(n) + \sum m_i$.

3. Is it admissible? Give a general proof or a concrete counterexample:
 - This is admissible because every mismatched spot has to be fixed to be the same as $goal[i]$. Each action changes one letter and so each action is different for each different spot. The word that action enters has $goal[i]$ in position i which means rarity is at least m_i and it costs at least $1 + m_i$. Adding all the mismatched spots up shows that each path ends up costing at least $h_R(n)$ which means it doesn't overestimate the cost. The cost is at least h_0 because every $m_i \geq 0$ as all m_i are positive.
4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative Not invalid.
5. Test the verified/repaired heuristic against h_0 on the two frequency-cost problems from Q2 (WHY $\rightarrow$ ASK, BEAR $\rightarrow$ DUCK). Compare solution costs and nodes expanded, then conclude whether the evidence supports using it.

Problem	Heuristic	Solution cost	Nodes expanded
WHY $\rightarrow$ ASK	h_0	38.86	755
WHY $\rightarrow$ ASK	new	38.86	510
BEAR $\rightarrow$ DUCK	h_0	38.24	720
BEAR $\rightarrow$ DUCK	new	38.24	319

- Conclusion: Both of the heuristics ended up returning the same optimal solution costs as it was 38.86 for WHY to ASK and 38.24 for BEAR to DUCK so switching to h_R didn't change anything. Also, h_R expanded fewer nodes as it was 510 as compared to 755 when doing WHY to ASK as well as 319 compared to 720 when doing BEAR to DUCK. So, as h_R is admissible as proved in part 3 the evidence shows it should be used rather than h_0 .

- Note: passing tests does not prove admissibility; the proof in item 3/4 is what counts.